Name: Temilade A. Oyebisi

Mat. No.: 259044030

Course: MEG831 Power Plant Project Report.

OPTIMIZATION OF AN AD – HTC FUEL ENHANCED GAS CYCLE.

This is a combination of Anaerobic Digestion – Hydrothermal Carbonization Pathway for Methane-Enhanced Power Generation.

Table of contents

1.0. Introduction

Objective:

To simulate the thermal and chemical behavior of a slurry in a hydrothermal carbonization reactor.

- To predict the internal temperature profile of the reactor sides and core.
- To monitor the "Conversion" (yield progress) of the biomass.
- To optimize parameters like flow velocity and wall temperature in real-time.

Core Technology:

- Streamlit Transforms the Python script into a web dashboard.
- NumPy Handles the heavy lifting of matrix calculations (the reactor grid).
- Matplotlib Visualizes the heat and chemical yield maps.
- SciPy (minimize): The "Optimizer" it mathematically searches for the preferred settings.

Key Functionality:

It solves a simplified version of the Heat Equation to show how temperature evolves over time and space within the reactor.

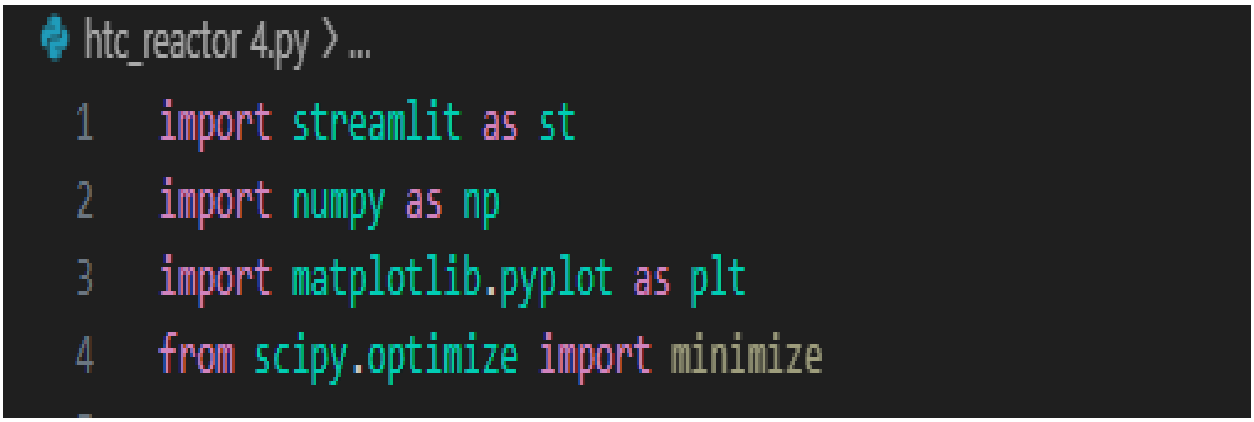

Fig. 1: Shows the codebase set-up.

2.0. SIMULATION

2.1. MODELLING HEAT & MASS TRANSFER

The vectorized_step function being the core logic calculates four physical phenomena simultaneously:

2.1.1. Advection (Flow): Heat moving along with the fluid velocity

2.1.2. Diffusion (Conduction): Heat spreading through the fluid based on thermal diffusivity.

2.1.3. Exothermic Reaction: Once the temperature exceeds 180°C, the code triggers a reaction that generates extra heat.

2.1.4. Yield Progress: A "Conversion Mask" tracks where the reaction has occurred, ensuring it only happens until 100% conversion is reached.

2.2. SIMULATION GRID

2.2.1. Reactor Space Detail.

- Dimensions: The reactor is modeled as a 2D grid (80 x 30 nodes).

2.2.2. Geometry:

- Length L = 4.0 meters
- Radius R = 0.6 meters

2.2.3. Boundary Conditions:

- Inlet: Forced temperature Tf
- Walls: Constant wall temperature (Tw) mimicking external heating jackets.
- Outlet: "Zero-gradient" boundary, meaning heat flows out naturally with the fluid.

2.2.4. Advection Fluid Flow:

This represents forced convection. It calculates how movement of the slurry (at velocity **u**) transfers heat downstream.

- It uses a "backward difference- comparing a cell to the one behind it.
- Where fluid is moving fast, heat is "pushed" toward the outlet more quickly.

2.2.5. Induction- Thermal Conduction

This models how heat naturally spreads out from hot areas to cold areas through the material itself, governed by the thermal diffusivity

2.2.6. Reaction Logic for the HTC

This is a simplified chemical model:

- The Trigger: The reaction only starts if the temperature is above 180°C and the material isn't fully converted yet (conv < 1.0).
- Heat Generation: If the reaction is active, it adds a constant heat source (5.0 units), simulating an exothermic reaction.
- Conversion: It increments the "yield" by 1% per time step.

2.2.7. Time step Integration plus setting boundary conditions

This is the **Euler integration**. It updates the temperature for the next tiny slice of time

- Temperature increases with diffusion and reaction heat.
- Temperature decreases as colder fluid is "advected" into that space.
- The wall boundary as well as the inlet and oulet boundaries are set

```
# Code written by TEMILADE A. OYEBISI
# MSc MECHANICAL ENGINEERING STUDENT, UNILAG



import streamlit as st
import numpy as np
import matplotlib.pyplot as plt

# --- Page Config & Styling ---
st.set_page_config(page_title="TEMILADE HTC Reactor Lab", layout="wide")
st.title("🌿 HTC Reactor: TEMILADE 259044030 LAYOUT")

# --- Constants & Physics ---
nx, ny = 100, 50
L, R = 8.0, 0.8
dx, dy = L/nx, R/ny
dt = 0.005  # Simulation time step (seconds)
rho_slurry = 1100.0  # Density of slurry (kg/m^3)
area = np.pi * (R**2)  # Reactor cross-section (m^2)

# Initialize state if not present
if 'T' not in st.session_state:
    st.session_state.T = np.ones((nx, ny)) * 40.0
    st.session_state.conv = np.zeros((nx, ny))
    st.session_state.ch4 = np.zeros((nx, ny))
    st.session_state.history = [] # For noting yield per simulation

def vectorized_step(T, conv, ch4, u, a, tw, tin):
    T_new = T.copy()
    conv_new = conv.copy()
    ch4_new = ch4.copy()

    # Physics calculations (Advection-Diffusion)
    adv_x = u * (T[1:-1, 1:-1] - T[:-2, 1:-1]) / dx
    diff_x = a * (T[2:, 1:-1] - 2*T[1:-1, 1:-1] + T[:-2, 1:-1]) / dx**2
    diff_y = a * (T[1:-1, 2:] - 2*T[1:-1, 1:-1] + T[1:-1, :-2]) / dy**2

    # --- ACTIVE KINETICS ---
    rxn_mask = (T[1:-1, 1:-1] > 180.0) & (conv[1:-1, 1:-1] < 1.0)

    # Hydrochar Conversion (Solid phase)
    conv_rate = 0.01 * (T[1:-1, 1:-1] / 200.0)
    conv_new[1:-1, 1:-1] += np.where(rxn_mask, conv_rate * dt * 10, 0.0)

    # Methane Yield (Gas phase byproduct)
    gas_mask = (T[1:-1, 1:-1] > 200.0) & (ch4[1:-1, 1:-1] < 0.25)

    # --- VARYING METHANE YIELD LOGIC ---
    thermal_yield_rate = 0.005 * (T[1:-1, 1:-1] / 200.0)**2
    active_conversion_state = conv[1:-1, 1:-1]
    dynamic_yield = 0.025 * (active_conversion_state * (1.0 -
active_conversion_state))

    total_yield_rate = thermal_yield_rate + dynamic_yield
```

```
    ch4_new[1:-1, 1:-1] += np.where(gas_mask, total_yield_rate * dt * 10,
0.0)

    # Heat balancing
    s_rxn = np.where(rxn_mask, 6.0, 0.0)
    T_new[1:-1, 1:-1] = T[1:-1, 1:-1] + dt * (-adv_x + diff_x + diff_y +
s_rxn)

    # Boundary Conditions
    T_new[0, :] = tin
    T_new[-1, :] = T_new[-2, :]
    T_new[:, 0] = tw
    T_new[:, -1] = tw

    return T_new, conv_new, ch4_new

def render_plots(T_data, conv_data, ch4_data, container):
    fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(10, 12))

    im1 = ax1.imshow(T_data.T, extent=[0, L, 0, R], origin='lower',
cmap='magma', aspect='auto', vmin=40, vmax=320)
    ax1.set_title("Temperature Profile (°C)")
    plt.colorbar(im1, ax=ax1)

    im2 = ax2.imshow(conv_data.T, extent=[0, L, 0, R], origin='lower',
cmap='Greens', aspect='auto', vmin=0, vmax=1)
    ax2.set_title("Hydrochar Conversion Yield")
    plt.colorbar(im2, ax=ax2)

    im3 = ax3.imshow(ch4_data.T, extent=[0, L, 0, R], origin='lower',
cmap='Blues', aspect='auto', vmin=0, vmax=0.20)
    ax3.set_title("Active Methane (CH₄) Gaseous Yield")
    plt.colorbar(im3, ax=ax3)

    plt.tight_layout()
    container.pyplot(fig)
    plt.close(fig)

# --- Sidebar Controls ---
st.sidebar.header("🕹️ Reactor Controls")
u_vel = st.sidebar.slider("Slurry Velocity [m/s]", 0.01, 1.0, 0.15)
t_wall = st.sidebar.slider("Wall Temp [°C]", 180.0, 360.0, 250.0)
alpha = st.sidebar.slider("Thermal Diffusivity", 0.001, 0.05, 0.01)
t_inlet = st.sidebar.number_input("Slurry Inlet Temp [°C]", value=40.0)
sim_steps = st.sidebar.number_input("Simulation Iterations", min_value=100,
max_value=5000, value=1000, step=50)
mass_flow_rate = rho_slurry * area * u_vel

col_btn1, col_btn2 = st.sidebar.columns(2)

# --- Main Display Area ---
plot_spot = st.empty()
metrics_spot = st.empty()

# RUN LOGIC 1: Animated Simulation
if col_btn1.button("▶️ Live Simulation"):
```

```
    st.session_state.T = np.ones((nx, ny)) * 40.0
    st.session_state.conv = np.zeros((nx, ny))
    st.session_state.ch4 = np.zeros((nx, ny))

    with st.spinner("Processing thermochemical reactions (Animated)..."):
        for i in range(sim_steps):
            st.session_state.T, st.session_state.conv, st.session_state.ch4 =
vectorized_step(
                st.session_state.T, st.session_state.conv,
st.session_state.ch4, u_vel, alpha, t_wall, 40.0
            )
            if i % 100 == 0:
                render_plots(st.session_state.T, st.session_state.conv,
st.session_state.ch4, plot_spot)

    # Calculation for the Log
    avg_ch4_yield = np.mean(st.session_state.ch4) * 100
    st.session_state.history.append({
        "Run Type": "Animated",
        "Wall Temp (°C)": t_wall,
        "Velocity (m/s)": u_vel,
        "Thermal Diffusivity": alpha,
        "Avg Methane Yield (%)": round(avg_ch4_yield, 4)
    })
    render_plots(st.session_state.T, st.session_state.conv,
st.session_state.ch4, plot_spot)

# RUN LOGIC 2: Instant Compute Option
if col_btn2.button("⚡ RUN"):
    st.session_state.T = np.ones((nx, ny)) * 40.0
    st.session_state.conv = np.zeros((nx, ny))
    st.session_state.ch4 = np.zeros((nx, ny))

    with st.spinner("Calculating final state..."):
        for i in range(sim_steps):
            st.session_state.T, st.session_state.conv, st.session_state.ch4 =
vectorized_step(
                st.session_state.T, st.session_state.conv,
st.session_state.ch4, u_vel, alpha, t_wall, 40.0
            )

    avg_ch4_yield = np.mean(st.session_state.ch4) * 100
    st.session_state.history.append({
        "Run Type": "Instant",
        "Wall Temp (°C)": t_wall,
        "Velocity (m/s)": u_vel,
        "Thermal Diffusivity": alpha,
        "Avg Methane Yield (%)": round(avg_ch4_yield, 4)
    })
    render_plots(st.session_state.T, st.session_state.conv,
st.session_state.ch4, plot_spot)


# Current Metrics Display
peak_t = np.max(st.session_state.T)
current_avg_ch4 = np.mean(st.session_state.ch4) * 100
```

```
with metrics_spot.container():
    m1, m2, m3, m4 = st.columns(4)
    m1.metric("Mass Flow Rate", f"{mass_flow_rate:.2f} kg/s")
    m2.metric("Peak Temp", f"{peak_t:.1f} °C")
    m3.metric("Avg Conversion", f"{np.mean(st.session_state.conv)*100:.1f}
%")
    m4.metric("Avg CH4 Yield", f"{current_avg_ch4:.4f} %")

# --- Simulation History ---
st.markdown("---")
st.subheader("📋 Methane Yield (Per Run)")
if st.session_state.history:
    st.table(st.session_state.history)
else:
    st.info("No runs recorded yet. Start a simulation to populate the yield
log.")

# --- Thermodynamic & Gas Analysis ---
st.markdown("---")
if st.button("📊 Analyse"):
    st.markdown("### 📄Thermodynamic Report")

    # 1. Mass Flow & Report Text
    avg_ch4 = 50  # example percentage
    gas_kg_s = (avg_ch4/100) * mass_flow_rate
    st.success(f"**Calculated System Mass Flow Rate:** {mass_flow_rate:.1f}
kg/s")
    st.info(f"""
    * **Calculated Methane Mass Flow:** {gas_kg_s:.5f} kg/s
    * **Reactor Efficiency:** At {t_wall}°C, the methane generation is
concentrated near the reactor walls.
    * **Thermal Sustainability:** Heat required to maintain this yield:
{(mass_flow_rate * 4.18 * (peak_t-40)):.1f} W.
    """)

    # 2. Thermodynamic Plots
    T_min_k = 40.0 + 273.15
    T_max_k = peak_t + 273.15

    fig_thermo, (ax_ts, ax_th) = plt.subplots(1, 2, figsize=(14, 5))

    # T-S Diagram (Steam Cycle approximation)
    # Using stylized coordinates to represent Pump -> Heating -> Boiling ->
Superheat -> Condenser
    s_points = [1.0, 2.5, 6.0, 7.5, 1.0]
    t_points = [T_min_k, 373.15, 373.15, T_max_k, T_min_k]

    ax_ts.plot(s_points, t_points, 'o-', color='crimson', linewidth=2,
label='Ideal Steam Cycle')
    ax_ts.fill(s_points, t_points, color='crimson', alpha=0.1)
    ax_ts.set_xlabel("Entropy (s) [kJ/kg·K]")
    ax_ts.set_ylabel("Temperature (T) [K]")
    ax_ts.set_title("T-S Diagram (Steam Phase)")
    ax_ts.grid(True, linestyle='--', alpha=0.6)
    ax_ts.legend()
```

```
    # T-H Diagram (Gas Phase / Methane Heating approximation)
    # Enthalpy H = Cp * T (Cp for CH4 is roughly 2.22 kJ/kgK)
    cp_ch4 = 2.22
    temps_th = np.linspace(T_min_k, T_max_k, 50)
    h_th = cp_ch4 * (temps_th - T_min_k) # Enthalpy change relative to inlet

    ax_th.plot(temps_th, h_th, '-', color='teal', linewidth=3, label='$CH_4$
Enthalpy Curve')
    ax_th.fill_between(temps_th, 0, h_th, color='teal', alpha=0.1)
    ax_th.set_xlabel("Temperature (T) [K]")
    ax_th.set_ylabel("Enthalpy Change (Δh) [kJ/kg]")
    ax_th.set_title("T-H Diagram")
    ax_th.grid(True, linestyle='--', alpha=0.6)
    ax_th.legend()

    st.pyplot(fig_thermo)

     # --- Embedded Report ---
    st.markdown("## 📄 Thermodynamic Analysis Report: TEMILADE 259044030")

    rep_col1, rep_col2 = st.columns(2)

    with rep_col1:
        st.info("#### 1. T-S (Temperature-Entropy) Analysis")
        st.markdown(f"""
        The T-S diagram represents the **Steam Rankine Cycle** coupled with
the HTC reactor.
        * **Isentropic Compression:** Effective pumping into the pressure
vessel is indicated by little entropy change.
        * **Heat Addition Phase:** Depicts the phase transition that takes
place at a constant temperature.
        * **Process Insight:** At the current peak temperature of
**{peak_t:.1f}°C**, Significant energy availability for possible heat
recovery is indicated by the entropy increase.
        """)

    with rep_col2:
        st.info("#### 2. T-H (Temperature-Enthalpy) Analysis")
        st.markdown(f"""
        The T-H diagram tracks the **Total Energy Content** of the process
gases.
        * **Linear Correlation:** Enthalpy ($h$) is a direct function of $T$
where $h = c_p T$.
        * **Energy Density:** The specific heat capacity is represented by
the slope. Successful exothermic reaction heat release is indicated by a
steep ascent.
        * **Efficiency:** The delta between inlet ($T_{{in}}$) and outlet
($T_{{out}}$) defines the net thermal gain.
        """)
```

3.0. Discussion of Simulation

- Slower feed velocity increases the conversion of the biomass into hydrochar, HTC thermal fluid and methane gas while faster feed velocity decreases the conversion rate of the biomass.
- The higher the Wall Temperature the better the conversion and more methane gas is produced but care must be taken not to exceed the critical peak temperature
- The higher the thermal diffusivity the better the conversion
- Producing biomass requires a temperature above 200 degree Celsius
- Heat required to maintain this yield can also be optimized from this model